

# INFO 239

## Comunicaciones

Instituto de Informática  
Facultad de Ciencias de la Ingeniería  
Universidad Austral de Chile

Guía de Trabajos Prácticos de Laboratorio

Dr. Ing. Christian Lazo Lamírez  
UACH - 2026

# Índice

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>Introducción general</b>                                                     | <b>2</b>  |
| <b>1. Unidad 1 — Sistemas de Comunicaciones</b>                                 | <b>3</b>  |
| 1.1. Experimento 1: Análisis y Simulación de Señales AC y DC en Python . . . .  | 3         |
| 1.1.1. Descripción . . . . .                                                    | 3         |
| 1.1.2. Marco teórico . . . . .                                                  | 3         |
| 1.1.3. Procedimiento . . . . .                                                  | 4         |
| 1.2. Experimento 1: Radio Galena . . . . .                                      | 7         |
| 1.2.1. Descripción . . . . .                                                    | 7         |
| 1.2.2. Marco teórico . . . . .                                                  | 7         |
| 1.2.3. Procedimiento . . . . .                                                  | 8         |
| 1.3. Experimento 2: Telégrafo Digital con Arduino . . . . .                     | 10        |
| 1.3.1. Descripción . . . . .                                                    | 10        |
| 1.3.2. Procedimiento . . . . .                                                  | 10        |
| <b>2. Unidad 2 — Capa Física</b>                                                | <b>13</b> |
| 2.1. Experimento 3: Atenuación en Cables . . . . .                              | 13        |
| 2.1.1. Descripción . . . . .                                                    | 13        |
| 2.1.2. Procedimiento . . . . .                                                  | 13        |
| 2.2. Experimento 4: Enlace Inalámbrico RF 433 MHz . . . . .                     | 15        |
| 2.2.1. Descripción . . . . .                                                    | 15        |
| 2.2.2. Procedimiento . . . . .                                                  | 15        |
| 2.3. Experimento 5: Enlace por Fibra Óptica Plástica e Interceptación . . . . . | 17        |
| 2.3.1. Descripción . . . . .                                                    | 17        |
| 2.3.2. Marco teórico . . . . .                                                  | 17        |
| 2.3.3. Procedimiento . . . . .                                                  | 17        |
| <b>3. Unidad 3 — Capa de Enlace de Datos</b>                                    | <b>20</b> |
| 3.1. Experimento 6: Implementación de CRC con Arduino . . . . .                 | 20        |
| 3.1.1. Descripción . . . . .                                                    | 20        |
| 3.1.2. Procedimiento . . . . .                                                  | 20        |
| 3.2. Experimento 7: Protocolos ARQ — Stop-and-Wait y Ventana Deslizante . .     | 22        |
| 3.2.1. Descripción . . . . .                                                    | 22        |
| 3.2.2. Marco teórico . . . . .                                                  | 22        |
| 3.2.3. Procedimiento . . . . .                                                  | 22        |
| 3.3. Experimento 8: Telégrafo Inalámbrico con ESP32 y WiFi . . . . .            | 25        |
| 3.3.1. Descripción . . . . .                                                    | 25        |
| 3.3.2. Procedimiento . . . . .                                                  | 25        |
| <b>4. Proyecto Integrador Final</b>                                             | <b>28</b> |
| <b>Resumen de experimentos y materiales</b>                                     | <b>30</b> |

## Introducción general

Esta guía reúne los trabajos prácticos del curso de Comunicaciones de Datos, organizados en tres unidades que siguen la progresión natural del modelo TCP/IP: desde la señal física hasta la capa de enlace de datos, con una introducción al direccionamiento IP al final del curso.

Cada Laboratorio está diseñado con tres capas de aprendizaje simultáneas:

1. **Capa técnica:** comprensión del fenómeno o protocolo mediante implementación real con software o hardware.
2. **Capa de seguridad:** identificación de vulnerabilidades propias de cada medio o protocolo, y análisis de contramedidas.
3. **Capa de IA:** uso crítico de herramientas de inteligencia artificial generativa como auxiliar de análisis, generación de código y documentación técnica.

## Plataformas y herramientas

| Herramienta                      | Uso principal                                 | Unidad  |
|----------------------------------|-----------------------------------------------|---------|
| Arduino Uno                      | Codificación, señales, protocolos enlace      | 1, 2, 3 |
| ESP32                            | WiFi, servidor/cliente TCP                    | 3       |
| Módulos RF 433MHz                | Enlace inalámbrico sin cifrado                | 2, 3    |
| Fibra óptica plástica (POF)      | Medio guiado óptico                           | 2       |
| Wireshark                        | Captura y análisis de tráfico                 | 2, 3    |
| Packet Tracer                    | Simulación de redes                           | 3       |
| Google Colab (Python)            | Análisis de señales, gráficos                 | 1       |
| IA generativa (Claude / ChatGPT) | Generación de código, análisis, documentación | 1, 2, 3 |

Cuadro 1: Resumen de herramientas por unidad

## Modalidad de trabajo

Los experimentos se realizan en grupos de 3 a 4 estudiantes. Cada grupo dispone de un kit completo de materiales. Al finalizar cada experimento, el grupo entrega un **informe en formato IEEE** que incluye introducción, marco teórico, desarrollo, resultados, análisis y conclusiones.

## 1 Unidad 1 — Sistemas de Comunicaciones

### 1.1 Experimento 1: Análisis y Simulación de Señales AC y DC en Python

**Semana 2** | **Vínculo:** señales analógicas y digitales, período, frecuencia, modelo de Shannon | **Duración:** 3 horas

#### 1.1.1 Descripción

En este laboratorio se desarrolla un programa en Python que simula y visualiza señales eléctricas alternas (AC) y continuas (DC), permitiendo modificar parámetros como amplitud, frecuencia y duración. Adicionalmente, se simula el efecto de un capacitor sobre una señal rectificada, observando cómo suaviza las variaciones y estabiliza la salida.

El trabajo se realiza en Google Colab, utilizando las bibliotecas `numpy` y `matplotlib`. Este experimento vincula directamente la teoría de señales vista en clases (parámetros de una señal sinusoidal, período, frecuencia, modelo de Shannon) con su representación gráfica y programática.

**Enlace al notebook base:**

<https://colab.research.google.com/drive/1lYAZEukU24yCQZDHHrEfGkRqSomNnv1?usp=sharing>

#### Objetivos del experimento

- Simular y visualizar señales eléctricas AC (senoidal) y DC (constante) usando Python.
- Modificar parámetros como amplitud, frecuencia y duración a través de variables del programa.
- Comparar señales AC con distintas frecuencias y observar el efecto del período.
- Simular la rectificación de media onda y onda completa, y el efecto de un capacitor de filtrado.
- Usar IA generativa para generar código Python, auditarlo y corregir errores.

#### 1.1.2 Marco teórico

Una señal sinusoidal de corriente alterna se describe como:

$$v(t) = A \sin(2\pi ft + \varphi) \quad (1)$$

donde  $A$  es la amplitud (voltios),  $f$  la frecuencia (Hz),  $t$  el tiempo (s) y  $\varphi$  la fase inicial (radianes). El período es  $T = 1/f$ .

Una señal de corriente continua (DC) es simplemente una constante:  $v(t) = V_{DC}$ .

La **rectificación de media onda** elimina los semiciclos negativos:  $v_r(t) = \max(0, v(t))$ . La **rectificación de onda completa** invierte los semiciclos negativos:  $v_r(t) = |v(t)|$ .

El **capacitor de filtrado** suaviza la señal rectificada almacenando carga durante los picos y entregándola durante las caídas. El efecto se modela mediante la ecuación diferencial discreta:

$$v_{cap}[i] = v_{cap}[i-1] + \frac{\Delta t}{\tau} (v_{rect}[i] - v_{cap}[i-1]), \quad \tau = RC \quad (2)$$

### 1.1.3 Procedimiento

#### Parte 1 — Señales AC y DC básicas

1. Abrir el notebook base en Google Colab (enlace arriba) o crear uno nuevo.
2. Definir las variables de configuración al inicio del notebook y generar las señales:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # ---- Parametros modificables ----
5 amplitud    = 5.0      # Voltios (pico)
6 frecuencia  = 50.0     # Hz
7 fase        = 0.0      # Radianes
8 nivel_dc    = 3.0      # Voltios (senal DC)
9 duracion    = 0.1      # Segundos a visualizar
10 fs          = 10000    # Muestras por segundo
11 # -----
12
13 t          = np.linspace(0, duracion, int(fs * duracion))
14 v_ac       = amplitud * np.sin(2 * np.pi * frecuencia * t + fase)
15 v_dc       = np.full_like(t, nivel_dc)

```

Listing 1: Variables de configuracion y generacion de senales — Python

3. Generar el **Gráfico 1**: señal AC y DC superpuestas. Incluir líneas verticales indicando el período  $T$ , leyenda, etiquetas de ejes y un cuadro de texto con los parámetros (amplitud, frecuencia, período).
4. Generar el **Gráfico 2**: dos señales AC con distinta frecuencia ( $f_1$  y  $f_2 = 2f_1$ ) superpuestas en el mismo eje para comparar visualmente el efecto del período.

#### Parte 2 — Rectificación y capacitor

1. Calcular las señales rectificadas de media onda y onda completa.
2. Implementar el modelo del capacitor de filtrado y generar el **Gráfico 3**:

```

1 # Rectificacion de media onda
2 v_media = np.where(v_ac > 0, v_ac, 0)
3
4 # Rectificacion de onda completa
5 v_completa = np.abs(v_ac)
6
7 # Modelo simplificado de capacitor
8 R = 1000 # Ohm
9 C = 100e-6 # Faradios
10 tau = R * C
11 dt = t[1] - t[0]
12
13 v_cap = np.zeros_like(v_completa)
14 v_cap[0] = v_completa[0]
15 for i in range(1, len(t)):
16     # Completar con ayuda de IA (ver actividad IA)
17     pass
18
19 # Grafico 3: rectificada y salida con capacitor
20 fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
21 axes[0].plot(t, v_completa, label='Onda completa rectificada')
22 axes[0].set_ylabel('Voltaje (V)')
23 axes[0].legend()
24 axes[1].plot(t, v_cap, color='green',
25             label='Con capacitor (tau={:.1f} ms)'.format(tau * 1000))
26 axes[1].set_ylabel('Voltaje (V)')
27 axes[1].set_xlabel('Tiempo (s)')
28 axes[1].legend()
29 plt.tight_layout()
30 plt.show()

```

Listing 2: Rectificacion y modelo de capacitor — Python

3. Experimentar con distintos valores de  $R$  y  $C$ : ¿qué ocurre cuando  $\tau$  es muy pequeño? ¿y muy grande?

#### Material por kit

| Herramienta / Recurso | Cantidad | Especificación                             |
|-----------------------|----------|--------------------------------------------|
| Computador o laptop   | 1        | Con acceso a Internet                      |
| Cuenta Google         | 1        | Para usar Google Colab                     |
| Notebook base         | 1        | Enlace en la descripción                   |
| Bibliotecas Python    | —        | numpy, matplotlib (preinstaladas en Colab) |

#### Seguridad e Implicancias

Una simulación incorrecta puede enmascarar fenómenos reales. El caso más relevante es el **aliasing**: si la frecuencia de muestreo  $f_s$  es inferior a  $2f_{max}$  (criterio de Nyquist), la señal reconstruida tendrá frecuencias falsas que no existen en la original. En sistemas de

comunicaciones reales, el aliasing puede generar interferencias o pérdida de información. Discutir: en el notebook, ¿con qué valor mínimo de `fs` se vería correctamente una señal de 50 Hz? ¿Qué pasaría si se usara `fs = 80`?

### Actividad con Inteligencia Artificial

#### Modo: Generador + Auditor

##### Actividad 1 — Completar el modelo de capacitor:

Copiar el código incompleto del bucle `for` y solicitar a la IA:

*“Completa este bucle en Python que modela la carga y descarga de un capacitor sobre una señal rectificada, usando la ecuación:  $v_{cap}[i] = v_{cap}[i-1] + (dt/tau) * (v_{rect}[i] - v_{cap}[i-1])$ ”*

Verificar que el resultado sea correcto: la señal de salida debe subir hasta el pico de la señal rectificada y descender lentamente entre picos. Si la IA comete un error (índice incorrecto, uso de `tau` equivocado), documentarlo y corregirlo.

##### Actividad 2 — Cuadro de texto en el gráfico:

Solicitar a la IA:

*“Genera el código Python para agregar un cuadro de texto al gráfico con los valores calculados de amplitud, frecuencia y período.”*

Auditar si el código usa `ax.text()` con los valores de las variables (no escritos a mano). Identificar al menos un error antes de incorporarlo al notebook.

### Preguntas de análisis y discusión

1. ¿Qué diferencias observas entre las señales AC y DC en el Gráfico 1? Menciona al menos tres características distintas.
2. ¿Cómo afecta la frecuencia a la forma de la señal AC? ¿Qué ocurre con el período al duplicar la frecuencia?
3. ¿Qué ocurriría si la amplitud de la señal AC aumentara mucho más que el nivel DC? ¿Qué consecuencia práctica tendría en un sistema real?
4. ¿Cómo cambia la señal con el uso del capacitor? Relaciona el valor de  $\tau = RC$  con la calidad del filtrado observada en el Gráfico 3.
5. Relaciona este experimento con el teorema de Nyquist: si  $f = 50$  Hz, ¿cuál es la frecuencia de muestreo mínima necesaria? ¿Qué valor usaron en su notebook y por qué?

## 1.2 Experimento 1: Radio Galena

**Semana 2** | **Vínculo:** señales analógicas, resonancia LC, modelo de Shannon | **Duración:** 3 horas

### 1.2.1 Descripción

La radio galena es el receptor de radio más simple posible: no requiere ninguna fuente de energía externa. Toda la energía para producir audio proviene de la onda de radio capturada por la antena. Este experimento hace tangible el concepto de que una señal electromagnética transporta energía suficiente para ser demodulada directamente.

El circuito sintoniza una frecuencia de la banda AM mediante un circuito resonante LC (bobina + condensador variable), detecta la envolvente de la señal modulada en amplitud mediante un diodo de germanio y la convierte en audio audible en un auricular de alta impedancia.

#### Objetivos del experimento

- Comprender el concepto de señal analógica y modulación en amplitud (AM).
- Observar el principio de resonancia LC como mecanismo de sintonización.
- Relacionar la energía de una señal de RF con la capacidad de transmitir información a distancia.
- Identificar las implicancias de seguridad de un medio de transmisión no cifrado y de acceso libre.

### 1.2.2 Marco teórico

La modulación en amplitud (AM) superpone la señal de información sobre una portadora de alta frecuencia variando su amplitud. La frecuencia de la portadora determina el canal de radio. El circuito LC sintoniza una frecuencia  $f_0$  según:

$$f_0 = \frac{1}{2\pi\sqrt{LC}} \quad (3)$$

donde  $L$  es la inductancia de la bobina (en henrios) y  $C$  es la capacitancia del condensador variable (en faradios). Al variar  $C$ , se modifica  $f_0$  y se sintoniza una estación diferente.

El diodo de germanio actúa como detector de envolvente: rectifica la señal AM y el condensador filtra la portadora, dejando solo la señal de audio de baja frecuencia.

### 1.2.3 Procedimiento

1. **Construcción de la bobina:** Enrollar 80 vueltas de hilo de cobre esmaltado calibre 26 sobre un tubo de PVC de 3 cm de diámetro. Dejar toma intermedia en la vuelta 15 desde tierra.
2. **Armado del circuito:** Conectar la bobina en paralelo con el condensador variable para formar el circuito resonante LC. El punto de unión se conecta al ánodo del diodo 1N34A.
3. **Detector y auricular:** Conectar el cátodo del diodo a un condensador de  $0,01 \mu\text{F}$  a tierra (filtro de portadora) y al auricular de alta impedancia ( $2000 \Omega$ ).
4. **Antena:** Conectar un cable de 10–15 metros a la toma intermedia de la bobina. Extenderlo horizontal o verticalmente.
5. **Sintonización:** Girar lentamente el condensador variable hasta escuchar una estación AM local.
6. **Medición:** Registrar el valor aproximado de  $C$  en el que se sintoniza cada estación. Calcular la inductancia  $L$  de la bobina y comparar con la frecuencia real de la emisora.

#### Materiales por kit

| Componente              | Cantidad | Especificación             |
|-------------------------|----------|----------------------------|
| Diodo de germanio       | 1        | 1N34A                      |
| Condensador variable    | 1        | 365 pF                     |
| Auricular               | 1        | 2000 $\Omega$              |
| Hilo de cobre esmaltado | 3 m      | Calibre 26 AWG             |
| Tubo PVC                | 1        | 3 cm diámetro, 10 cm largo |
| Condensador cerámico    | 1        | 0,01 $\mu\text{F}$         |
| Cable antena            | 15 m     | Cable fino                 |
| Protoboard y jumpers    | 1 set    | —                          |

#### Seguridad e Implicancias

La banda AM es un medio completamente abierto: cualquier receptor puede capturar cualquier transmisión sin autorización ni equipos especiales. Este experimento demuestra empíricamente que **interceptar una señal de radio no requiere romper ningún cifrado**, simplemente construir un receptor. Discutir: ¿Qué tipo de información debería transmitirse por medios abiertos como AM? ¿Qué consecuencias tendría transmitir datos sensibles por este canal?

### Actividad con Inteligencia Artificial

Consultar a una IA generativa: “*Explica por qué la radio AM puede ser recibida sin fuente de energía externa y qué vulnerabilidades de seguridad tiene este medio.*”

Evaluar críticamente la respuesta: ¿Es técnicamente correcta? ¿Omite algo importante? ¿Comete errores conceptuales? Documentar al menos dos imprecisiones o aciertos relevantes en el informe.

### Preguntas de análisis y discusión

1. ¿De dónde proviene la energía que alimenta el auricular si no hay batería en el circuito?
2. ¿Qué ocurre con la sintonización si duplicas el número de vueltas de la bobina?
3. ¿Por qué se usa un diodo de germanio (1N34A) y no uno de silicio (1N4007)?
4. Relaciona este experimento con el teorema de Shannon: ¿cuál sería el canal y cuál el ruido en este sistema?
5. ¿Qué debería agregar al circuito para que la comunicación fuera bidireccional?

## 1.3 Experimento 2: Telégrafo Digital con Arduino

**Semana 4** | **Vínculo:** codificación digital, señal binaria, sincronización  
| **Duración:** 3 horas

### 1.3.1 Descripción

Dos Arduinos se comunican mediante una señal digital: uno lee un pulsador y codifica la entrada en código Morse, el otro decodifica y muestra el mensaje en el monitor serial o en un display LCD. El estudiante observa directamente la señal binaria y experimenta con el concepto de temporización, sincronización y codificación.

#### Objetivos del experimento

- Comprender la representación binaria de información y la codificación por tiempo.
- Experimentar la diferencia entre transmisión síncrona y asíncrona.
- Observar la señal digital en el monitor serial como representación del tren de bits.
- Usar IA para generar código, auditarlo y corregir errores lógicos.

### 1.3.2 Procedimiento

1. Conectar un pulsador al pin digital 2 del Arduino transmisor con resistencia pull-down de 10 k $\Omega$ .
2. Cargar el código de codificación Morse (generado con ayuda de IA, ver actividad IA).
3. Conectar el pin TX del Arduino transmisor al pin RX del receptor (con nivel lógico compatible o divisor de tensión si es necesario).
4. El Arduino receptor decodifica la señal y muestra el resultado en el monitor serial a 9600 bps.
5. Transmitir al menos 5 palabras y verificar la recepción correcta.
6. Introducir deliberadamente un error de timing (modificar la constante de tiempo del punto/raya) y observar el efecto en la decodificación.

```
1 // Constantes de tiempo Morse (en milisegundos)
2 #define PUNTO    100
3 #define RAYA     300
4 #define ESP_SIM  100 // espacio entre simbolos
5 #define ESP_LET  300 // espacio entre letras
6
7 const int PIN_TX = 1; // Pin de transmision serial
8 const int PIN_BTN = 2; // Pin del pulsador
9
10 // Tabla Morse: completar con ayuda de IA
```

```

11 // Ejemplo: A = .- , B = -...
12 void setup() {
13     Serial.begin(9600);
14     pinMode(PIN_BTN, INPUT);
15 }
16
17 void loop() {
18     // TODO: leer pulsador y codificar en Morse
19     // Actividad: pedir a la IA que complete esta funcion
20     // Luego identificar si el codigo generado es correcto
21 }

```

Listing 3: Transmisor Morse — esquema base (completar con IA)

### Materiales por kit

| Componente            | Cantidad | Especificación    |
|-----------------------|----------|-------------------|
| Arduino Uno           | 2        | —                 |
| Pulsador              | 1        | 12 mm             |
| Resistencia           | 2        | 10 k $\Omega$     |
| LCD 16×2 + módulo I2C | 1        | Opcional          |
| Cable jumper          | varios   | Macho-macho       |
| Protoboard            | 1        | —                 |
| Cable USB             | 2        | Para programación |

### Seguridad e Implicancias

Este sistema transmite en texto plano a través de un canal eléctrico. Cualquier persona con acceso físico al cable puede leer la señal con un multímetro o un osciloscopio. Discutir: ¿qué se necesitaría agregar para que la comunicación fuera confidencial? Introducir el concepto de cifrado como capa adicional sobre la codificación.

### Actividad con Inteligencia Artificial

Solicitar a una IA: *“Escribe el código Arduino completo para un telégrafo Morse: el transmisor lee un pulsador y el receptor muestra el texto decodificado por serial.”*

Auditar el código generado: ¿compila sin errores?, ¿la temporización es correcta?, ¿maneja correctamente los espacios entre letras y palabras? Documentar los errores encontrados y las correcciones realizadas.

### Preguntas de análisis y discusión

1. ¿Qué ocurre si el transmisor y receptor tienen velocidades de reloj ligeramente diferentes? ¿Cómo se resuelve en protocolos reales?
2. ¿Cuántos bits por segundo transmite este sistema usando la constante de tiempo definida?

3. ¿Es este sistema síncrono o asíncrono? ¿Por qué?
4. Relaciona este experimento con el modelo OSI: ¿en qué capa opera el código Morse?

## 2 Unidad 2 — Capa Física

### 2.1 Experimento 3: Atenuación en Cables

**Semana 6** | **Vínculo:** atenuación, diafonía, categorías de cable, normas EIA/TIA | **Duración:** 3 horas

#### 2.1.1 Descripción

Un Arduino genera una señal cuadrada a frecuencia conocida que se transmite a través de diferentes tipos de cable. En el extremo receptor, otro Arduino o un multímetro mide la tensión resultante. Los estudiantes grafican la atenuación en función de la frecuencia y la longitud, y contrastan sus resultados con las especificaciones de los estándares EIA/TIA-568.

##### Objetivos del experimento

- Medir experimentalmente la atenuación en distintos tipos de cable (UTP cat3, cat5e, cat6, coaxial).
- Relacionar la atenuación con la frecuencia de la señal y la longitud del cable.
- Contrastar resultados experimentales con especificaciones del estándar EIA/TIA-568.
- Usar Python (Google Colab) con ayuda de IA para graficar y analizar los resultados.

#### 2.1.2 Procedimiento

1. Generar señales cuadradas con el Arduino a frecuencias de 1 kHz, 10 kHz, 100 kHz y 1 MHz (usando la función `tone()` o PWM).
2. Transmitir cada señal por tramos de cable de 1 m, 5 m y 10 m de cada tipo: UTP cat3, cat5e, cat6 y cable de timbre (referencia).
3. Medir la tensión pico a pico en el extremo receptor con multímetro y registrar en tabla.
4. Calcular la atenuación en dB:  $A = 20 \cdot \log_{10}(V_{out}/V_{in})$ .
5. Ingresar los datos en Google Colab y generar gráficos de atenuación vs frecuencia y atenuación vs longitud con ayuda de IA.
6. Comparar con los límites máximos de atenuación del estándar TIA-568-C para cada categoría.

## Materiales por kit

| Componente               | Cantidad | Especificación         |
|--------------------------|----------|------------------------|
| Arduino Uno              | 2        | —                      |
| UTP cat3                 | 10 m     | Tramos de 1, 5 y 10 m  |
| UTP cat5e                | 10 m     | Tramos de 1, 5 y 10 m  |
| UTP cat6                 | 10 m     | Tramos de 1, 5 y 10 m  |
| Cable coaxial RG58       | 10 m     | Referencia             |
| Multímetro digital       | 1        | Con función AC         |
| Resistencias 75 $\Omega$ | 4        | Terminación de línea   |
| Protoboard y jumpers     | 1 set    | —                      |
| Laptop con Google Colab  | 1        | Para análisis de datos |

## Seguridad e Implicancias

Los cables de mayor categoría no solo atenúan menos: también tienen mayor trenzado, lo que reduce la diafonía (NEXT) y las emisiones electromagnéticas externas. Este fenómeno tiene implicancias de seguridad: un cable con alta emisión electromagnética puede ser “escuchado” a distancia sin contacto físico. Investigar el concepto de **TEMPEST** y cómo las normas de blindaje (STP, FTP) mitigan este riesgo.

## Actividad con Inteligencia Artificial

Proporcionar los datos medidos a una IA con el siguiente prompt: “Aquí están mis mediciones de atenuación en cables UTP [tabla de datos]. Genera el código Python para graficar atenuación vs frecuencia para cada categoría de cable y añade los límites del estándar TIA-568-C.”

Verificar que el código generado sea correcto, los ejes estén bien etiquetados y los límites del estándar sean precisos. Corregir lo que sea necesario.

## Preguntas de análisis y discusión

1. ¿A qué frecuencia comienza a ser significativa la diferencia entre cat5e y cat6 en tus mediciones?
2. ¿Por qué el cable de timbre tiene mayor atenuación que el UTP cat3?
3. ¿Qué relación existe entre el número de vueltas del trenzado y la inmunidad al ruido?
4. Si tuvieras que cablear una sala de servidores con equipos de alta seguridad, ¿qué tipo de cable elegirías y por qué?

## 2.2 Experimento 4: Enlace Inalámbrico RF 433 MHz

**Semana 7** | **Vínculo:** medios no guiados, atenuación en espacio libre, interceptación | **Duración:** 3 horas

### 2.2.1 Descripción

Dos Arduinos con módulos RF 433 MHz establecen un enlace inalámbrico y transmiten mensajes de texto. Los estudiantes miden el efecto de la distancia, los obstáculos y la orientación de la antena en la calidad del enlace. Luego, un tercer módulo receptor intercepta la comunicación sin autorización, demostrando la vulnerabilidad inherente de los medios no guiados sin cifrado.

#### Objetivos del experimento

- Establecer un enlace de datos inalámbrico funcional usando módulos RF 433 MHz.
- Medir la degradación de la señal en función de la distancia y los obstáculos.
- Demostrar la interceptación pasiva de una comunicación no cifrada.
- Comprender la necesidad del cifrado en comunicaciones inalámbricas.

### 2.2.2 Procedimiento

1. Conectar el módulo transmisor RF al pin digital 12 del Arduino transmisor. Conectar el receptor RF al pin digital 11 del Arduino receptor. Instalar la biblioteca `RadioHead` en el IDE de Arduino.
2. Cargar el sketch de transmisión en el primer Arduino y el de recepción en el segundo. Verificar comunicación a distancia de 1 m.
3. Medir la tasa de paquetes recibidos correctamente a distancias de 1, 5, 10, 20 y 50 m en espacio abierto. Registrar en tabla.
4. Repetir la medición con obstáculos: pared de tabique, pared de hormigón, puerta metálica.
5. **Interceptación:** conectar un tercer módulo receptor RF (del kit de otro grupo) a un tercer Arduino. Sin modificar el enlace original, verificar que el tercer nodo recibe todos los mensajes.
6. Discutir: ¿qué debería agregarse al sistema para evitar esta interceptación?

## Materiales por kit

| Componente           | Cantidad | Especificación                  |
|----------------------|----------|---------------------------------|
| Arduino Uno          | 3        | (el tercero puede ser prestado) |
| Módulo TX RF 433 MHz | 1        | XD-FST o similar                |
| Módulo RX RF 433 MHz | 2        | XD-RF-5V o similar              |
| Antena de hilo       | 3        | 17,3 cm para 433 MHz            |
| Cinta métrica        | 1        | 50 m                            |
| Protoboard y jumpers | 2 sets   | —                               |

## Seguridad e Implicancias

Este experimento demuestra que **cualquier receptor en la misma frecuencia puede recibir cualquier transmisión**, sin importar a quién iba dirigida. Los módulos RF 433 MHz no tienen ningún mecanismo de cifrado ni autenticación. Esto es equivalente a una red WiFi abierta sin WPA. Introducir el concepto de confidencialidad como propiedad de seguridad que debe ser agregada explícitamente a la capa de enlace.

## Actividad con Inteligencia Artificial

Consultar a una IA: *“Tengo un enlace de comunicación con módulos RF 433 MHz sin cifrado. ¿Qué opciones tengo para agregar confidencialidad e integridad a las transmisiones usando un Arduino Uno?”*

Evaluar las opciones propuestas: ¿son implementables en Arduino?, ¿consumen demasiada memoria?, ¿cuál elegirías para un prototipo real? Documentar el análisis.

## Preguntas de análisis y discusión

1. ¿Cómo varía la potencia de la señal recibida con la distancia? ¿Se corresponde con la ley del cuadrado inverso?
2. ¿Qué obstáculo produjo mayor atenuación? ¿Por qué?
3. ¿Cuántos receptores simultáneos pueden escuchar el mismo transmisor? ¿Hay algún límite?
4. ¿Qué diferencia hay entre este enlace RF y una red WiFi en términos de seguridad?

## 2.3 Experimento 5: Enlace por Fibra Óptica Plástica e Interceptación

**Semana 7** | **Vínculo:** fibra óptica, reflexión interna total, seguridad física | **Duración:** 3 horas

### 2.3.1 Descripción

Los estudiantes construyen un enlace de datos usando fibra óptica plástica (POF), modulando un LED con una señal digital de Arduino y detectando la luz en el otro extremo con un fototransistor. Luego intentan interceptar la señal mediante tres técnicas: curvatura forzada (macrobending), empalme artesanal y detección pasiva sin contacto.

#### Objetivos del experimento

- Construir un enlace de comunicación óptico usando POF, LED y fototransistor.
- Comprender el principio de reflexión interna total y cómo se rompe en el macrobending.
- Demostrar experimentalmente la interceptación de fibra por curvatura forzada.
- Comparar la interceptabilidad de la fibra óptica con el enlace RF 433 MHz del Experimento 4.

### 2.3.2 Marco teórico

La propagación de luz en una fibra óptica se basa en la **reflexión interna total**: cuando la luz viaja desde un medio de mayor índice de refracción ( $n_1$ , núcleo) a uno menor ( $n_2$ , revestimiento), y el ángulo de incidencia supera el ángulo crítico  $\theta_c$ , la luz se refleja completamente sin pérdidas de transmisión:

$$\theta_c = \arcsin\left(\frac{n_2}{n_1}\right) \quad (4)$$

Al doblar la fibra en un radio pequeño, los rayos de luz inciden sobre la interfaz a un ángulo menor que  $\theta_c$ , escapando del núcleo. Esta fuga de luz puede ser captada por un sensor externo, constituyendo una técnica real de interceptación.

### 2.3.3 Procedimiento

#### Parte 1 — Construcción del enlace

1. Cortar un tramo de 2 m de fibra POF con un corte limpio y perpendicular (usar cúter caliente o herramienta de corte).

2. Conectar el LED rojo al pin digital 9 del Arduino transmisor con resistencia de  $220\ \Omega$ .
3. Acoplar el extremo del LED a la fibra usando cinta negra para evitar pérdidas de luz ambiental.
4. Conectar el fototransistor al extremo receptor con resistencia de  $10\ \text{k}\Omega$  al pin analógico A0.
5. Transmitir una señal PWM desde el Arduino y verificar recepción leyendo el pin analógico. El valor debe cambiar con la señal transmitida.
6. Agregar comunicación serial simple: transmitir texto en ASCII y mostrarlo en el monitor serial del receptor.

### **Parte 2 — Interceptación por macrobending**

1. Con el enlace activo transmitiendo texto, doblar la fibra alrededor de un lápiz (radio  $\approx 3\ \text{mm}$ ) en el punto medio del recorrido.
2. Acercar un segundo fototransistor al punto de curvatura. Conectar este sensor al pin analógico A0 de un tercer Arduino.
3. Verificar si el sensor interceptor detecta la señal. Registrar el valor ADC con y sin curvatura.
4. Reducir el radio de curvatura hasta obtener la mayor señal posible en el interceptor sin cortar el enlace.

### **Parte 3 — Comparación con RF 433 MHz**

1. Transmitir el mismo mensaje por el enlace POF y por el enlace RF 433 MHz del Experimento 4.
2. Completar la tabla comparativa de la sección de análisis.

## Materiales por kit

| Componente                  | Cantidad | Especificación             |
|-----------------------------|----------|----------------------------|
| Fibra óptica plástica (POF) | 2–3 m    | 2,2 mm de diámetro         |
| LED rojo 5 mm               | 2        | Alta luminosidad           |
| Fototransistor              | 3        | LTR-4206 o 3DU5C           |
| Arduino Uno                 | 3        | (uno del kit vecino)       |
| Resistencia 220 $\Omega$    | 2        | Para LED                   |
| Resistencia 10 k $\Omega$   | 3        | Pull-down fototransistor   |
| Aceite de bebé              | 1        | Acoplador óptico artesanal |
| Lápiz o cilindro 3 mm       | 1        | Para macrobending          |
| Cinta negra opaca           | 1        | Sellado óptico             |
| Protoboard y jumpers        | 1 set    | —                          |

## Seguridad e Implicancias

La fibra óptica **no emite campos electromagnéticos** detectables externamente, lo que la hace más segura que el cobre ante escucha remota. Sin embargo, este experimento demuestra que **no es invulnerable**: requiere acceso físico directo al cable, pero una vez que hay acceso, la interceptación por macrobending es posible y no interrumpe el enlace. Esto implica que la seguridad de la fibra descansa en la seguridad física del recorrido del cable, y que la detección de intrusiones puede hacerse monitoreando la potencia óptica en el receptor (una caída indica posible interceptación).

## Actividad con Inteligencia Artificial

Solicitar a una IA que genere una tabla comparativa entre fibra óptica y RF 433 MHz en las siguientes dimensiones: interceptabilidad, requerimiento de acceso físico, detección de la interceptación, costo, distancia máxima, velocidad máxima y uso de cifrado nativo. Verificar cada celda con los resultados experimentales y con Stallings. Corregir lo que sea incorrecto.

## Preguntas de análisis y discusión

1. ¿A qué radio de curvatura mínimo lograste detectar señal en el interceptor? ¿Por qué un radio mayor no funciona?
2. ¿Cómo podría un administrador de red detectar que alguien está interceptando su fibra por macrobending?
3. ¿En qué escenarios reales se preferiría fibra óptica sobre cobre o inalámbrico desde el punto de vista de la seguridad?
4. ¿Qué ocurre con el aceite de bebé como acoplador en el empalme? ¿Por qué mejora la transmisión?

## 3 Unidad 3 — Capa de Enlace de Datos

### 3.1 Experimento 6: Implementación de CRC con Arduino

**Semana 9** | **Vínculo:** detección de errores, CRC, integridad de datos  
| **Duración:** 3 horas

#### 3.1.1 Descripción

Un Arduino transmite tramas de datos con un código CRC-8 calculado. El receptor verifica la integridad de cada trama. Se introducen errores deliberados modificando bits en el código, y los estudiantes observan que el CRC los detecta. Se compara la eficacia del CRC con la paridad simple.

##### Objetivos del experimento

- Implementar el cálculo de CRC-8 en Arduino y verificarlo en el receptor.
- Demostrar que el CRC detecta errores de 1, 2 y 3 bits mientras que la paridad simple falla ante errores de número par.
- Distinguir entre detección de errores (CRC) e integridad criptográfica (hash).
- Auditar código generado por IA para encontrar errores lógicos en la implementación.

#### 3.1.2 Procedimiento

1. Usar la IA para generar el código de cálculo CRC-8 con polinomio generador  $x^8 + x^2 + x + 1$  (0x07).
2. Auditar el código generado antes de cargarlo (ver actividad IA).
3. Cargar el transmisor: envía tramas de 8 bytes + 1 byte de CRC por serial.
4. Cargar el receptor: verifica el CRC de cada trama recibida e indica si es correcta o errónea.
5. Introducir errores: modificar un bit en la trama en el código del transmisor y observar que el receptor detecta el error.
6. Implementar también la paridad simple y demostrar que falla ante errores de 2 bits.
7. Registrar: ¿cuántos errores de bits detecta el CRC-8? ¿Puede corregirlos?

```
1 // Polinomio generador: 0x07 (x^8 + x^2 + x + 1)
2 uint8_t calcularCRC8(uint8_t *data, uint8_t len) {
3     uint8_t crc = 0x00;
4     for (uint8_t i = 0; i < len; i++) {
```

```

5   crc ^= data[i];
6   for (uint8_t bit = 0; bit < 8; bit++) {
7       // TODO: completar con ayuda de IA
8       // Luego verificar si el algoritmo es correcto
9   }
10  }
11  return crc;
12 }

```

Listing 4: Función CRC-8 — completar con IA y auditar

### Material por kit

| Componente             | Cantidad | Especificación             |
|------------------------|----------|----------------------------|
| Arduino Uno            | 2        | —                          |
| Cable jumper           | varios   | Para conexión TX/RX        |
| Resistencia 1 kΩ       | 2        | Divisor de tensión 5V/3.3V |
| Laptop con Arduino IDE | 1        | —                          |

### Seguridad e Implicancias

El CRC detecta errores accidentales pero **no protege contra modificaciones intencionales**: un atacante puede recalcular el CRC correcto para una trama modificada. Introducir la diferencia con los **hash criptográficos** (SHA-256, HMAC): estos están diseñados para ser computacionalmente imposibles de falsificar sin la clave. Esto ilustra por qué integridad de datos en seguridad requiere criptografía, no solo CRC.

### Actividad con Inteligencia Artificial

Solicitar a la IA el código CRC-8 completo. Antes de cargarlo, revisarlo buscando: errores en el orden de las operaciones XOR, condición incorrecta en el bucle de bits, valor inicial del CRC incorrecto. Registrar si la IA cometió algún error y cómo lo detectaron. Luego preguntar a la IA: “¿Por qué el CRC no es suficiente para garantizar integridad en un contexto de seguridad?” y evaluar la respuesta.

### Preguntas de análisis y discusión

1. ¿Qué ocurre si se invierten 2 bits en la misma posición de bytes diferentes? ¿El CRC lo detecta?
2. ¿Puede el CRC-8 corregir errores o solo detectarlos? ¿Qué se necesitaría para corrección?
3. ¿Por qué un atacante puede burlar el CRC pero no puede burlar un HMAC-SHA256?
4. ¿En qué capa del modelo TCP/IP opera el CRC de Ethernet? ¿Y el checksum de TCP?

## 3.2 Experimento 7: Protocolos ARQ — Stop-and-Wait y Ventana Deslizante

**Semana 10** | **Vínculo:** control de flujo, ARQ, eficiencia del enlace | **Duración:** 3 horas

### 3.2.1 Descripción

Usando comunicación serial entre dos Arduinos (o vía RF 433 MHz para introducir pérdidas reales), los estudiantes implementan primero el protocolo Stop-and-Wait y miden su throughput efectivo. Luego modifican el código para simular una ventana deslizante de tamaño 3 y comparan la eficiencia. La IA genera el código base de Stop-and-Wait, que los estudiantes extienden.

#### Objetivos del experimento

- Implementar el protocolo ARQ Stop-and-Wait y medir su throughput real.
- Comparar la eficiencia de Stop-and-Wait con una ventana deslizante de tamaño 3.
- Relacionar los resultados con la fórmula teórica de eficiencia de cada protocolo.
- Extender código base generado por IA para implementar ventana deslizante.

### 3.2.2 Marco teórico

La eficiencia de Stop-and-Wait se define como:

$$\eta_{sw} = \frac{1}{1 + 2a} \quad (5)$$

donde  $a = t_p/t_f$  es la relación entre el tiempo de propagación  $t_p$  y el tiempo de transmisión de una trama  $t_f$ .

Para una ventana deslizante de tamaño  $W$ :

$$\eta_W = \begin{cases} 1 & \text{si } W \geq 2a + 1 \\ \frac{W}{2a + 1} & \text{si } W < 2a + 1 \end{cases} \quad (6)$$

### 3.2.3 Procedimiento

1. Cargar el código Stop-and-Wait (generado con IA): transmisor envía una trama y espera ACK antes de enviar la siguiente. Receptor envía ACK al recibir cada trama

correctamente.

2. Transmitir 100 tramas de 32 bytes y medir el tiempo total. Calcular el throughput en bytes/segundo.
3. Introducir pérdidas artificiales: el receptor descarta aleatoriamente el 10 % de las tramas (usando `random()`). Medir cómo afecta al throughput.
4. Modificar el código para implementar ventana de tamaño 3: el transmisor envía hasta 3 tramas sin esperar ACK. Medir el throughput y comparar.
5. Calcular el valor teórico de  $\eta$  para cada protocolo usando la fórmula y comparar con los resultados medidos.

#### Material por kit

| Componente             | Cantidad | Especificación                  |
|------------------------|----------|---------------------------------|
| Arduino Uno            | 2        | —                               |
| Módulos RF 433 MHz     | 1 par    | Para introducir pérdidas reales |
| Laptop con Arduino IDE | 1        | —                               |
| Cable USB              | 2        | —                               |

#### Seguridad e Implicancias

Los protocolos ARQ garantizan **integridad y entrega** pero no **confidencialidad**: los ACKs y los números de secuencia están en texto plano. Un atacante puede realizar un **ataque de replay**: capturar una trama válida y reenviarla posteriormente, haciendo que el receptor la procese como si fuera nueva. Discutir cómo los protocolos modernos (TLS, QUIC) incorporan números de secuencia cifrados para prevenir esto.

#### Actividad con Inteligencia Artificial

Solicitar a la IA el código base de Stop-and-Wait. Luego pedir: *“Modifica este código para implementar una ventana deslizante de tamaño  $W=3$ .”*

Evaluar: ¿maneja correctamente los ACKs fuera de orden?, ¿implementa el timeout correctamente?, ¿funciona cuando se pierden tramas? Documentar los problemas encontrados y las correcciones realizadas.

#### Preguntas de análisis y discusión

1. Con los valores medidos, ¿qué tamaño de ventana necesitarías para alcanzar el 90 % de eficiencia en este enlace?
2. ¿Qué ocurre si el receptor envía un ACK negativo (NACK) en lugar de simplemente no responder?
3. ¿Por qué Go-Back-N puede ser menos eficiente que Selective Repeat cuando hay muchos errores?

4. Relaciona este experimento con el control de congestión de TCP: ¿en qué se parece la ventana deslizante al concepto de ventana de congestión?

### 3.3 Experimento 8: Telégrafo Inalámbrico con ESP32 y WiFi

**Semana 13** | **Vínculo:** 802.11, proceso de asociación, WPA2 vs AP abierto | **Duración:** 3 horas

#### 3.3.1 Descripción

Los estudiantes retoman el telégrafo del Experimento 2 pero ahora usando ESP32 vía WiFi. Un ESP32 actúa como Access Point y servidor TCP, el otro como cliente. Transmiten mensajes de texto y con Wireshark en un laptop observan el tráfico primero en un AP abierto, luego con WPA2. El contraste entre ambas capturas hace visible el valor del cifrado.

#### Objetivos del experimento

- Implementar un enlace de datos sobre WiFi usando ESP32 en modo AP y cliente TCP.
- Capturar y analizar el tráfico generado con Wireshark.
- Comparar el tráfico visible en un AP abierto versus WPA2.
- Relacionar el Experimento 4 (RF sin cifrado) con la problemática de las redes WiFi abiertas.

#### 3.3.2 Procedimiento

##### Parte 1 — AP abierto

1. Programar el primer ESP32 como Access Point sin contraseña (`WiFi.softAP("TestNet")`) y servidor TCP en el puerto 8080.
2. Programar el segundo ESP32 como cliente TCP que se conecta al AP y envía mensajes cada 2 segundos.
3. Conectar un laptop al AP abierto y capturar con Wireshark. Filtrar por la IP del ESP32 cliente.
4. Verificar que los mensajes enviados entre los ESP32 son completamente legibles en la captura de Wireshark.

##### Parte 2 — AP con WPA2

1. Modificar el AP para usar WPA2 (`WiFi.softAP("TestNet", clave1234)`).
2. Repetir la captura con Wireshark desde el laptop (sin conectarse al AP).
3. Verificar que ahora el tráfico aparece cifrado (solo tramas 802.11 con payload encriptado).

4. Conectar el laptop al AP con WPA2 y repetir la captura. Verificar que ahora sí puede ver el tráfico TCP.

```

1 #include <WiFi.h>
2
3 const char* ssid = "TestNet";
4 // Para AP abierto: no password
5 // Para WPA2: const char* password = "clave1234";
6
7 WiFiServer server(8080);
8
9 void setup() {
10   Serial.begin(115200);
11   // AP abierto: WiFi.softAP(ssid);
12   // AP WPA2: WiFi.softAP(ssid, password);
13   WiFi.softAP(ssid);
14   Serial.print("AP IP: ");
15   Serial.println(WiFi.softAPIP());
16   server.begin();
17 }
18
19 void loop() {
20   WiFiClient client = server.available();
21   if (client) {
22     while (client.connected()) {
23       if (client.available()) {
24         String msg = client.readStringUntil('\n');
25         Serial.println("Recibido: " + msg);
26       }
27     }
28   }
29 }

```

Listing 5: ESP32 como Access Point y servidor TCP

### Materiales por kit

| Componente                  | Cantidad | Especificación                 |
|-----------------------------|----------|--------------------------------|
| ESP32                       | 2        | Con antena integrada           |
| Cable USB-C o micro-USB     | 2        | Para programación              |
| Laptop con Wireshark        | 1        | Adaptador WiFi en modo monitor |
| Arduino IDE + soporte ESP32 | 1        | esp32 board package            |

### Seguridad e Implicancias

Este experimento conecta dos conceptos vistos en el curso: el Experimento 4 demostró que RF 433 MHz es completamente interceptable porque no tiene cifrado. Este experimento demuestra que **WiFi sin WPA2 tiene exactamente la misma vulnerabilidad**. El cifrado WPA2 resuelve la confidencialidad pero no la autenticación del

AP: un **Evil Twin** (AP falso con el mismo nombre) puede engañar a los clientes para que se conecten y envíen tráfico al atacante, incluso con WPA2 Personal. Introducir WPA3/SAE como solución a este problema.

### Actividad con Inteligencia Artificial

Mostrar a la IA la captura de Wireshark del AP abierto (como texto o descripción) y preguntar: *“Analiza esta captura de Wireshark de un AP WiFi abierto. ¿Qué información sensible es visible? ¿Cómo lo mitigarías a nivel de protocolo?”*

Comparar el análisis de la IA con lo que el grupo observó directamente en la captura. ¿La IA identificó todo lo relevante? ¿Agregó información incorrecta?

### Preguntas de análisis y discusión

1. ¿Por qué al conectarte al AP con WPA2 sí puedes ver el tráfico en Wireshark, pero desde afuera no?
2. ¿Qué información sigue siendo visible incluso en una red WPA2 (pista: tramas de gestión 802.11)?
3. ¿En qué se diferencia WPA2-Personal de WPA2-Enterprise? ¿Cuál usarías en una universidad?
4. Relaciona los 8 experimentos del curso: ¿cómo ha evolucionado la “facilidad de interceptación” desde la radio galena hasta el ESP32 con WPA2?

## 4 Proyecto Integrador Final

**Semana 15** | **Vínculo:** integración completa del curso | **Duración:** 6 horas (sesión completa + trabajo autónomo)

### Descripción

Cada equipo diseña e implementa una **red híbrida mínima funcional** que integra los tres medios trabajados durante el curso: un nodo sensor con RF 433 MHz (enlace no cifrado), un gateway ESP32 WiFi que recibe los datos del sensor y los reenvía, y un laptop con Wireshark que monitorea todo el tráfico. El equipo documenta el diseño, identifica vulnerabilidades en cada tramo y propone contramedidas.

### Arquitectura del proyecto

[Sensor Arduino + RF 433] --RF-->[Gateway ESP32 + WiFi] --WiFi-->[Servidor / Laptop]

- **Nodo sensor (Arduino + RF 433 MHz):** lee un valor analógico (potenciómetro o sensor de temperatura) y lo transmite cada 5 segundos en formato CSV sin cifrado: `id,timestamp,valor`.
- **Gateway ESP32:** recibe los datos RF, los muestra en el monitor serial y los reenvía por WiFi (TCP) a un servidor en el laptop. Actúa como puente entre dos medios físicos distintos.
- **Servidor en laptop:** un script Python recibe los datos TCP y los registra en un archivo CSV. Wireshark captura todo el tráfico WiFi simultáneamente.

### Entregables

1. **Informe técnico en formato IEEE** que incluya: descripción de la arquitectura, justificación de cada decisión de diseño, diagramas de la red y del flujo de datos, tabla de vulnerabilidades por capa con su contramedida propuesta, y análisis de las capturas de Wireshark.
2. **Presentación oral de 15 minutos** donde el equipo demuestra el sistema funcionando y defiende sus decisiones de diseño ante las preguntas del docente.
3. **Código fuente comentado** de los tres nodos (Arduino sensor, ESP32 gateway, script Python servidor).

## Tabla de vulnerabilidades — plantilla

| Tramo                | Vulnerabilidad identificada | Evidencia | Contramedida propuesta |
|----------------------|-----------------------------|-----------|------------------------|
| RF 433 MHz           |                             |           |                        |
| WiFi (AP)            |                             |           |                        |
| Datos en el servidor |                             |           |                        |
| Protocolo de enlace  |                             |           |                        |

Cuadro 2: Plantilla de análisis de vulnerabilidades por tramo

## Criterios de evaluación

| Criterio                   | Ponderación | Instrumento             |
|----------------------------|-------------|-------------------------|
| Funcionamiento del sistema | 30 %        | Demostración en vivo    |
| Calidad del informe IEEE   | 30 %        | Rúbrica de informe      |
| Análisis de seguridad      | 25 %        | Rúbrica de análisis     |
| Presentación oral          | 15 %        | Rúbrica de presentación |

Cuadro 3: Criterios de evaluación del proyecto integrador

## Resumen de experimentos y materiales

| N° | Experimento               | Semana | Unidad   | Contenido principal                                     |
|----|---------------------------|--------|----------|---------------------------------------------------------|
| 1  | Radio Galena              | 2      | Unidad 1 | Señales AM, resonancia LC, interceptación abierta       |
| 2  | Telégrafo Digital Arduino | 4      | Unidad 1 | Codificación digital, sincronización, señal binaria     |
| 3  | Atenuación en Cables      | 6      | Unidad 2 | Atenuación, diafonía, normas EIA/TIA-568                |
| 4  | Enlace RF 433 MHz         | 7      | Unidad 2 | Medios no guiados, interceptación inalámbrica           |
| 5  | Fibra Óptica POF          | 7      | Unidad 2 | Reflexión interna total, macrobending, seguridad física |
| 6  | CRC con Arduino           | 9      | Unidad 3 | Detección de errores, integridad vs criptografía        |
| 7  | ARQ Stop-and-Wait         | 10     | Unidad 3 | Control de flujo, eficiencia de enlace                  |
| 8  | ESP32 WiFi                | 13     | Unidad 3 | 802.11, WPA2, Wireshark, cifrado                        |
| –  | Proyecto Integrador       | 15     | Todas    | Red híbrida, análisis de seguridad completo             |

Cuadro 4: Resumen de todos los experimentos del curso

## Kit completo de materiales por grupo

| Componente                  | Cantidad | Experimento(s)    |
|-----------------------------|----------|-------------------|
| Arduino Uno                 | 3        | 1–7               |
| ESP32                       | 2        | 8, Proyecto       |
| Módulo TX RF 433 MHz        | 1        | 4, 6, 7, Proyecto |
| Módulo RX RF 433 MHz        | 2        | 4, 6, 7, Proyecto |
| Diodo germanio 1N34A        | 1        | 1                 |
| Condensador variable 365 pF | 1        | 1                 |
| Auricular 2000 $\Omega$     | 1        | 1                 |

| Componente                               | Cantidad | Experimento(s) |
|------------------------------------------|----------|----------------|
| Hilo de cobre esmaltado calibre 26 (3 m) | 1        | 1              |
| Tubo PVC 3 cm diámetro                   | 1        | 1              |
| Fibra óptica plástica POF (3 m)          | 1        | 5              |
| LED rojo 5 mm alta luminosidad           | 3        | 5, 8           |
| Fototransistor LTR-4206                  | 3        | 5              |
| LCD 16×2 + módulo I2C                    | 1        | 2              |
| Pulsador 12 mm                           | 2        | 2              |
| Resistencias surtidas                    | 20       | General        |
| Condensadores cerámicos surtidos         | 10       | General        |
| Protoboard 830 puntos                    | 2        | General        |
| Cable jumper macho-macho (40 unid.)      | 1 set    | General        |
| Cable USB tipo A-B                       | 3        | Arduino        |
| Cable USB-C o micro-USB                  | 2        | ESP32          |
| Cable UTP cat3 (10 m)                    | 1        | 3              |
| Cable UTP cat5e (10 m)                   | 1        | 3              |
| Cable UTP cat6 (10 m)                    | 1        | 3              |
| Antenas de hilo 17,3 cm                  | 3        | 4, Proyecto    |
| Cinta negra opaca                        | 1        | 5              |
| Aceite de bebé (acoplador óptico)        | 1        | 5              |

Cuadro 5: Lista completa de materiales por kit de grupo